

Java API Collections e Streams

CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Agrupamento de objetos

Necessidade de uma lista com tamanho dinâmico

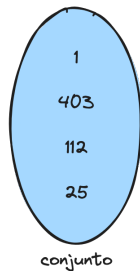
- Você precisa armazenar um grupo de objetos, por exemplo, uma lista de nomes de pessoas e realizar operações sobre eles, como adicionar, remover, buscar e ordenar os objetos armazenados
- Se optar por um vetor, terá que lidar com as seguintes limitações:
 - Tamanho fixo pode gerar desperdício de memória ou falta de espaço para novos elementos
 - Não oferece métodos para manipular os elementos

```
String [] nomes = new String[1000];  
nomes[0] = "Maria";  
nomes[1] = "Juca";  
nomes[2] = "Pedro";  
  
// Qual a melhor forma de remover o nome "Juca" do vetor?  
// Qual o pior caso para buscar um nome no vetor? Resposta: O(n)
```

Grupos naturais de objetos

Conjunto

- Um conjunto é um grupo de objetos que não tem uma ordem específica e não permite elementos duplicados
- Exemplo: Conjunto de números naturais $\{0, 1, 2, 3, \dots\}$

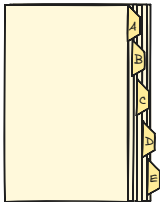


i Nota

Se usarmos um vetor para representar um conjunto, então toda vez que quisermos adicionar um elemento, teremos que verificar se ele já existe no vetor para evitar elementos duplicados

Agenda de contatos

Um mapa ou um dicionário



Lista de contatos

- Uma agenda de contatos é um grupo de objetos que tem uma ordem específica e permite elementos duplicados
- Uma agenda em papel possui abas para cada letra do alfabeto, o que facilita a busca por um contato específico

i Nota

Um vetor não oferece uma forma rápida de acessar um elemento específico, como um contato na agenda, pois é necessário percorrer o vetor para encontrar o elemento desejado

API Java Collections

- Trata-se de um conjunto de interfaces, classes e algoritmos para trabalhar com coleções de objetos em Java
- Oferece uma forma eficiente e fácil de usar para armazenar, obter e manipular dados agregados, como listas, conjuntos, filas e mapas

Java Collections Framework I

Interfaces e suas implementações

■ List

- Coleção ordenada e permite elementos duplicados, sendo possível acessar elementos por índice
- Implementações: ArrayList, LinkedList, Vector, Stack

■ Set

- Segue a abstração de conjuntos matemáticos e não permite elementos duplicados e não mantém a ordem de inserção
- Implementações: HashSet, LinkedHashSet, TreeSet

Java Collections Framework II

Interfaces e suas implementações

■ Queue

- Fila que ordena elementos para serem processados posteriormente, por exemplo, FIFO (primeiro que entra é o primeiro que sai)
- Implementações: `LinkedList`, `ArrayDeque`, `PriorityQueue`

■ Map

- Mapeia chaves para valores (também conhecido como dicionário)
- Não permite chaves duplicadas e cada chave mapeia somente um valor
- Implementações: `HashMap`, `LinkedHashMap`, `TreeMap`

Exemplos com coleções

ArrayList

Armazena objetos em um vetor, cujo tamanho aumenta automaticamente

```
// Criando uma lista de objetos da classe String
ArrayList<String> lista = new ArrayList<>();

// Método add adiciona um elemento no final da lista
lista.add("POO");
lista.add("IFSC");

String nome = lista.get(1); // Obtém elemento na posição 1
int total = lista.size(); // Obtém o total de elementos na coleção

// Criar uma lista a partir de um vetor
String[] nomes = {"Maria", "Juca", "Pedro"};
ArrayList<String> lista2 = new ArrayList<>(Arrays.asList(nomes));

// Criar um vetor a partir de uma lista
String[] vetor = lista.toArray(new String[0]);
```

ArrayList

Iterando sobre os elementos e removendo elementos

```
ArrayList<Pessoa> pessoas = new ArrayList<>();
pessoas.add(new Pessoa("Ana", 40));
pessoas.add(new Pessoa("Juca", 30));
pessoas.add(new Pessoa("Maria", 25));

for(Pessoa elemento: pessoas){ // Usando for-each para percorrer a lista
    System.out.println(elemento);
}

// Usando forEach com lambda
pessoas.forEach(elemento -> System.out.println(elemento));
// Usando forEach com referência de método
pessoas.forEach(System.out::println);

// Removendo elemento caso atenda a uma condição
pessoas.removeIf(elemento->elemento.getNome().equals("Juca"));
```

Set I

Coleção que não permite elementos duplicados

```
HashSet<String> conjuntoCidades = new HashSet<>();

conjuntoCidades.add("São José");
conjuntoCidades.add("Palhoça");
conjuntoCidades.add("Florianópolis");
conjuntoCidades.add("São José"); // Tenta adicionar o elemento "São José" novamente

// Imprime os elementos do conjunto.
// Note que "São José" aparece apenas uma vez, pois conjuntos não permitem elementos duplicados
conjuntoCidades.forEach(I0::println);

// Removendo todos elementos
conjuntoCidades.clear();
```

Set II

Coleção que não permite elementos duplicados

```
HashSet<String> conjuntoCidades = new HashSet<>();
conjuntoCidades.add("São José");
conjuntoCidades.add("Palhoça");
conjuntoCidades.add("Florianópolis");

// Criando um ArrayList a partir de um HashSet
ArrayList<String> listaCidades = new ArrayList<>(conjuntoCidades);

// Lista permite elementos duplicados, então "São José" pode ser adicionado novamente
listaCidades.add("São José");

// Criando um HashSet a partir de um ArrayList
HashSet<String> conjuntoCidades2 = new HashSet<>(listaCidades);

// Note que "São José" aparece apenas uma vez, pois conjuntos não permitem elementos duplicados
conjuntoCidades2.forEach(IO::println);
```

Queue

Coleção que ordena elementos para serem processados posteriormente, por exemplo, FIFO (primeiro que entra é o primeiro que sai)

```
Queue<String> fila = new LinkedList<>();
fila.add("Juca");
fila.add("Ana");
fila.add("Maria");

String nome = fila.poll(); // Remove o primeiro elemento da fila
if (nome != null) { // Nulo se a fila estiver vazia
    System.out.println(nome);
}
```

- Queue é uma interface que estende Collection, algumas implementações:
 - LinkedList
 - ArrayDeque
 - PriorityQueue

Map

Mapeia chaves para valores (*key, value*). Chaves são únicas e cada chave mapeia somente um valor

```
// chave: CPF, valor: Pessoa
HashMap<String, Pessoa> pessoas = new HashMap<>();
pessoas.put("123", new Pessoa("Juca", 35));
pessoas.put("456", new Pessoa("Maria", 20));
pessoas.put("789", new Pessoa("Ana", 31));

Pessoa p = pessoas.get("123"); // Obtém a pessoa associada à chave "123"
if (p != null) {
    System.out.println(p);
}

// Iterando sobre chaves, valores e pares chave-valor usando forEach com lambda
pessoas.forEach((cpf, pessoa) -> {
    System.out.println("CPF: " + cpf);
    System.out.println("Nome: " + pessoa.getNome());
    System.out.println("Idade: " + pessoa.getIdade());
});
```

Map

Iterando sobre chaves, valores e pares chave-valor

```
// Iterando sobre chaves
for (String cpf : pessoas.keySet()) {
    System.out.println("CPF: " + cpf);
}

// Iterando sobre valores
for (Pessoa pessoa : pessoas.values()) {
    System.out.println("Nome: " + pessoa.getNome());
    System.out.println("Idade: " + pessoa.getIdade());
}

// Iterando sobre pares chave-valor
for (Map.Entry<String, Pessoa> entry : pessoas.entrySet()) {
    System.out.println("CPF: " + entry.getKey());
    System.out.println("Nome: " + entry.getValue().getNome());
    System.out.println("Idade: " + entry.getValue().getIdade());
}
```


Map

Removendo elementos

■ Remoção por chave

```
HashMap<String, Pessoa> pessoas = new HashMap<>();  
pessoas.put("123", new Pessoa("Juca", 35));  
pessoas.put("456", new Pessoa("Maria", 20));  
  
// Remove a pessoa associada à chave "123"  
pessoas.remove("123");
```

■ Remoção por valor usando removeIf com lambda

```
HashMap<String, Pessoa> pessoas = new HashMap<>();  
pessoas.put("123", new Pessoa("Juca", 35));  
pessoas.put("456", new Pessoa("Maria", 20));  
  
// Remover todas as pessoas com o nome "Ana"  
pessoas.values().removeIf(pessoa -> pessoa.getNome().equals("Ana"));
```

Java Streams

Paradigma de programação funcional

- O foco está em descrever o que queremos alcançar (declarativo), em vez de como realizar uma tarefa (imperativo)
- Enfatiza o uso de funções puras, imutabilidade e expressões lambda para criar código mais conciso, legível e fácil de manter
- **Java Streams** é uma API para processar coleções de forma funcional

Stream em Java

- **Stream** é uma sequência de elementos processada por operações **intermediárias** encadeadas e finalizada por uma operação **terminal**
- **Não armazena dados**, apenas processa um fluxo proveniente de coleções, arrays, arquivos, etc
- **Não altera a fonte de dados**, mas retorna um novo fluxo com os resultados das operações
- Permite processamento sequencial ou paralelo

Exemplo de uso de Java Streams vs código imperativo

Deixar em maiúsculo o nome de todas as pessoas que iniciam com a letra "A"

```
List<String> nomes = Arrays.asList("Ana", "Juca", "Maria", "Amanda");
```

■ Exemplo de código imperativo

```
List<String> resultado = new ArrayList<>();  
for (String nome : nomes) {  
    if (nome.startsWith("A")) {  
        resultado.add(nome.toUpperCase());  
    }  
}
```

Exemplo de uso de Java Streams vs código imperativo

Deixar em maiúsculo o nome de todas as pessoas que iniciam com a letra "A"

```
List<String> nomes = Arrays.asList("Ana", "Juca", "Maria", "Amanda");
```

■ Exemplo de código imperativo

```
List<String> resultado = new ArrayList<>();  
for (String nome : nomes) {  
    if (nome.startsWith("A")) {  
        resultado.add(nome.toUpperCase());  
    }  
}
```

■ Exemplo de código usando Java Streams

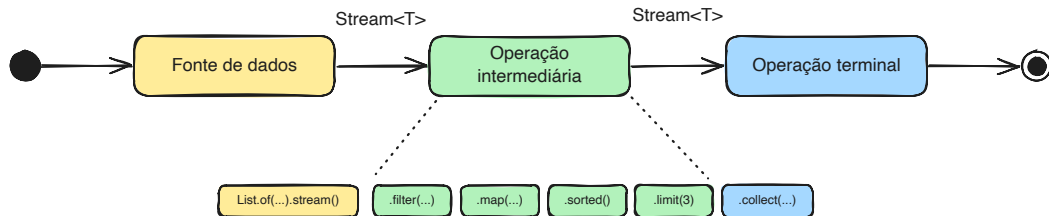
```
List<String> resultado =  
    nomes.stream()  
        .filter(nome -> nome.startsWith("A"))  
        .map(String::toUpperCase).toList();
```

Características do Java Streams

- **Sem armazenamento**, processa um fluxo de dados proveniente de uma fonte
- **Funcional por natureza**, operações intermediárias transformam o fluxo e retornam um novo fluxo
- **Consumível**, um fluxo só pode ser consumido uma vez, após a operação terminal, o fluxo é fechado e não pode ser reutilizado
- **Avaliação tardia** (*lazy evaluation*), a execução das operações intermediárias é adiada até que seu resultado seja necessário para a operação terminal, objetivando processamento eficiente

Componentes do fluxo de execução do Java Streams

- O fluxo de execução passa por uma ou mais operações intermediárias, que transformam os dados e retornam um novo fluxo, e termina em uma operação terminal, que produz um resultado ou efeito colateral
 - Efeito colateral: imprimir algo na tela, escrever em um arquivo, etc.



Operações intermediárias

Algumas operações intermediárias comuns em Java Streams

Sem estado (*stateless*)

Processam cada elemento **independentemente**

- `filter(Predicate)`
- `map(Function)`
- `flatMap(Function)`
- `mapToInt / Long / Double`
- `limit(n)`
- `skip(n)`

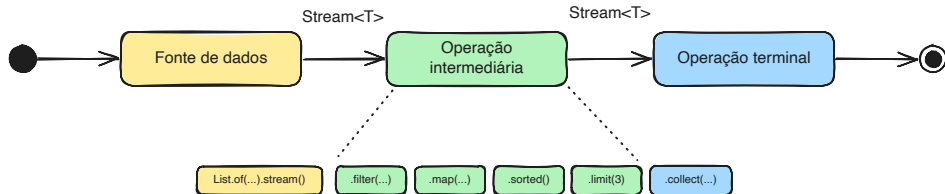
Com estado (*stateful*)

Precisam ver **elementos anteriores** para decidir

- `distinct()`
- `sorted(Comparator?)`

Podem exigir armazenamento adicional para manter o estado, o que pode afetar a eficiência do processamento com grandes volumes de dados

Componentes do fluxo de execução do Java Streams



```
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5, 6);  
  
numeros.stream() // Cria um fluxo de dados a partir da coleção  
    .filter(numero -> numero % 2 == 0) // Operação intermediária  
    .forEach(System.out::println); // Operação terminal
```

- Os dados fluem através de uma série de operações intermediárias, onde cada operação pode transformar os dados para depois serem processados por uma operação terminal

Alguns exemplos com Streams I

Operações intermediárias e terminais

- Somar os números pares de uma lista

```
List<Integer> lista = Arrays.asList(1, 2, , 4, 5, 6);
int soma = 0;

soma = lista.stream()
    .filter(numero -> numero % 2 == 0) // Filtra os números pares
    .mapToInt(Integer::intValue) // Converte para IntStream
    .sum(); // Soma os números

// Código imperativo equivalente
soma = 0;
for (Integer numero : lista) {
    if (numero % 2 == 0) {
        soma += numero;
    }
}
```

Alguns exemplos com Streams II

Operações intermediárias e terminais

- Obter o tamanho da maior palavra em uma lista de palavras

```
List<String> lista = Arrays.asList("Java", "Stream", "API", "Collections");

// Usando lambda
int maior = lista.stream()
    .mapToInt(p-> p.length()) // Converte cada palavra para seu comprimento (int)
    .max() // Encontra o valor máximo entre os comprimentos
    .orElse(0); // Retorna 0 se a lista estiver vazia

// Outra forma de escrever usando referência de método
int maior = lista.stream()
    .mapToInt(String::length) // Converte cada palavra para seu comprimento (int)
    .max() // Encontra o valor máximo entre os comprimentos
    .orElse(0); // Retorna 0 se a lista estiver vazia
```

Alguns exemplos com Streams III

Operações intermediárias e terminais

- Descobrir a pessoa com o nome mais longo em uma lista de pessoas

```
List<Pessoa> pessoas = Arrays.asList(new Pessoa("Ana", 40), new Pessoa("Juca", 30), new Pessoa("Maria", 25));
String nomeMaisLongo = pessoas.stream()
    .map(pessoa -> pessoa.getNome()) // Extrai os nomes das pessoas
    .max(Comparator.comparingInt(nome -> nome.length()))
    .orElse(""); // Retorna uma string vazia se a lista estiver vazia

// Outra forma de escrever usando referência de método
nomeMaisLongo = pessoas.stream().map(Pessoa::getNome)
    .max(Comparator.comparingInt(String::length)).orElse("");

// Código imperativo equivalente
String nomeMaisLongo = "";
for (Pessoa pessoa : pessoas) {
    if (pessoa.getNome().length() > nomeMaisLongo.length()) {
        nomeMaisLongo = pessoa.getNome();
    }
}
```

Alguns exemplos com Streams IV

Operações intermediárias e terminais

```
// Classe Pessoa com atributos nome, idade e país de origem
ArrayList<Pessoa> pessoas = new ArrayList<>();
pessoas.add(new Pessoa("Ana", 40, "Brasil"));
pessoas.add(new Pessoa("Juca", 30, "Brasil"));
pessoas.add(new Pessoa("Maria", 25, "Uruguai"));

Pessoa p = pessoas.stream()
    .filter(pessoa -> pessoa.getNome().equals("Maria")) // filtra
    .findFirst() // Retorna o primeiro elemento que atende a condição
    .orElse(null); // retorna null se não encontrar

if (p != null){ // se encontrou
    System.out.println(p);
}
```

Alguns exemplos com Streams V

Operações intermediárias e terminais

```
// Ordenando os elementos da lista por idade
pessoas.sort(Comparator.comparing(Pessoa::getIdade));

// Contando quantas pessoas tem idade maior que 30
long total = pessoas.stream()
    .filter(pessoa -> pessoa.getIdade() > 30) // filtra os elementos
    .count(); // Conta os elementos filtrados

// Contando quantas pessoas tem idade maior que 30 e país de origem Brasil
long totalBrasil = pessoas.stream()
    .filter(pessoa -> pessoa.getIdade() > 30) // filtra por idade
    .filter(pessoa -> pessoa.getPaisOrigem().equals("Brasil"))
    .count(); // Conta os elementos filtrados
```

Alguns exemplos com Streams VI

Operações intermediárias e terminais

```
// Montando uma lista com pessoas cujo país de origem seja "Brasil"
List<Pessoa> pessoasBrasil = pessoas.stream()
    .filter(pessoa -> pessoa.getPaisOrigem().equals("Brasil")).toList();

// Obtendo uma lista de nomes das pessoas com idade maior que 30
List<String> maioresDe30 = pessoas.stream()
    .filter(pessoa -> pessoa.getIdade() > 30)
    .map(Pessoa::getNome).toList(); // Coleta os nomes em uma nova lista

// Imprimindo os nomes das pessoas em caixa alta
pessoas.stream()
    .map(pessoa -> pessoa.getNome().toUpperCase())
    .forEach(System.out::println); // Exibe os nomes em caixa alta
```


Exercícios

Casamento de chaves, colchetes e parênteses

- Verifique se o casamento de parênteses, colchetes e chaves está correto em uma expressão matemática
- Exemplo: $-\{[b \times b - (4 \times a \times c)]/(2 \times a)\}$
- Algoritmo:
 - Use uma pilha para armazenar os símbolos de abertura
 - Para cada símbolo na expressão:
 - Se for um símbolo de abertura, empilhe-o
 - Se for um símbolo de fechamento, desempilhe o último símbolo de abertura e verifique se ele corresponde ao símbolo de fechamento
 - Se não corresponder ou se a pilha estiver vazia, a expressão é inválida
 - Se a pilha estiver vazia no final, a expressão é válida; caso contrário, é inválida

Referências



SILVA, Carlos. **Java 8: Iniciando o desenvolvimento com a Streams API**. 2016. Disponível em:
<https://www.oracle.com/br/technical-resources/articles/java-stream-api.html>.